École Nationale des Sciences Appliquées de Berrechid

Programme: Information Systems Engineering and Big Data

Introductory Internship Report

Design and Automation of a Data Platform for Insurance Analytics

Internship at Sanlam Maroc

Prepared by: RACHIDI ALAOUI Badr
Academic year: 2025–2026
Internship type: Introductory internship
Host entity: Data Factory
Duration: One month
Period: 4 August–2 September 2026
Academic supervisor: Pr FAKHAR
Company supervisor: Mr HAIDER Mohieddine

Casablanca | September 2026

Abstract

Insurance reporting depends on bringing together information that is recorded at different levels: customers, policies, premium transactions, claims and claim payments. Before these records can support a dashboard, they must be extracted consistently, checked against business rules and aggregated without double counting. This report presents the data platform prototype developed during a one-month introductory internship at Sanlam Maroc.

The project uses synthetic records generated in Oracle to protect customer confidentiality. A Python ingestion process captures new and updated rows and writes Parquet batches to MinIO. Apache Spark then builds a validated Silver layer and five business-oriented Gold datasets. Apache Airflow coordinates the workflow in a Docker environment, and the resulting CSV files support analysis in Tableau Desktop.

The work produced an automated preparation pipeline, reusable transformation modules, data-quality checks and a dashboard workbook. Validation covers the available exports and evidence retained from development. It demonstrates the approach on the synthetic dataset, rather than measuring Sanlam's business performance or certifying a production deployment. The report explains the design decisions, implementation and remaining limitations, including recovery across systems, memory requirements and consistent publication of related outputs.

Keywords: insurance analytics, data engineering, incremental ingestion, data lake, Apache Spark, Apache Airflow, Tableau.

Contents

Abstract i

General Introduction 1

1 Company Background and Project Context 2
- 1.1 Sanlam Maroc at a glance . . . 2
- 1.2 History and group context . . . 2
- 1.3 Internship setting and assignment . . . 4
- 1.4 Enterprise data architecture presented during the internship . . . 5
- 1.5 Problem statement, objectives and deliverables . . . 6
- 1.6 Data confidentiality and project boundaries . . . 7

2 State of the Art and Technical Foundations 8
- 2.1 From operational records to analytical datasets . . . 8
- 2.2 Analytical storage architectures . . . 8
- 2.3 Loading data and capturing changes . . . 9
- 2.4 Processing engines, orchestration and deployment . . . 10
- 2.5 Quality, traceability and the selected approach . . . 11

3 Requirements and Platform Design 12
- 3.1 Requirements analysis . . . 12
- 3.2 System architecture . . . 12
- 3.3 Source model and lake layers . . . 14
- 3.4 Incremental ingestion design . . . 16
- 3.5 Analytical output design . . . 18
- 3.6 Quality and publication design . . . 18

4 Implementation, Validation and Evaluation 20
- 4.1 Repository organisation . . . 20
- 4.2 Implementing the processing stages . . . 21
- 4.3 Connecting Spark to MinIO . . . 22
- 4.4 Airflow orchestration . . . 23
- 4.5 Docker runtime and reproducibility . . . 24
- 4.6 Visual evidence and analytical outputs . . . 25
- 4.7 Validation results and their scope . . . 30
- 4.8 Security and operational improvements . . . 31
- 4.9 Lessons from the implementation . . . 32

General Conclusion 33

Glossary 34

References 35

List of Figures

1.1 Selected milestones in the company's development. The milestones are shown in chronological order, with spacing that is not proportional to elapsed time. Sources: Sanlam Maroc and Allianz. . . . 3

1.2 SanlamAllianz markets and selected wider Sanlam Group locations, based on sources consulted on 11 September 2026. Colour indicates presence and organisational scope, not revenue, market share or customer density. Map data: Natural Earth[7]. . . . 4

3.1 Implemented data platform. Airflow coordinates ingestion, Spark transformation and CSV publication. Solid arrows show data movement; dashed arrows show orchestration dependencies. . . . 13

3.2 The nine-entity demonstration model. Cardinalities describe permitted relationships, and the relationship between a policy and an agent is optional. . . . 14

3.3 The lake's three quality layers: retained source batches, validated current records and business-oriented outputs. All three are stored in MinIO. . . . 16

3.4 A successful, non-empty ingestion. The Raw upload completes before Oracle commits the new watermark. Dashed message arrows are responses, not orchestration links. . . . 17

4.1 The thirteen-task Airflow workflow. Nine independent ingestions may run concurrently, subject to available worker capacity; Silver waits for all nine. . . . 23

4.2 A completed Airflow run showing source generation, ingestion, Spark transformations and CSV export. . . . 25

4.3 MinIO's sign-in interface for the local S3-compatible object store. . . . 25

4.4 The `sanlam-data-lake` bucket and its Raw, Silver and Gold prefixes. . . . 26

4.5 Raw prefixes corresponding to the nine Oracle business tables. . . . 26

4.6 Customer ingestion batches organised by `load_date`. This date identifies extraction, not the business event. . . . 27

4.7 The Silver layer with standardised, lower-case dataset names. . . . 27

4.8 A published Silver customer dataset, including its Parquet file and Spark success marker. . . . 28

4.9 Customer 360 dashboard: simulated risk-score distribution, customer segments and customers holding multiple policies. . . . 28

4.10 Policy dashboard: policy statuses, premiums by region, policy counts by product and premiums by product. . . . 29

4.11 Claims dashboard: claimed and approved amounts, claim types and statuses, payment years and simulated fraud flags. . . . 29

List of Tables

1.1 Selected Sanlam Maroc company figures from 2025 reporting 2

2.1 Storage approaches considered for the internship 9
2.2 Comparison of extraction strategies 10

3.1 Requirements and corresponding checks 12
3.2 Relationship between the enterprise concept and the implemented prototype 13
3.3 Gold datasets and their intended grains 18

4.1 Versions defined for the containerised Airflow execution path 24
4.2 Available synthetic CSV snapshot, inspected on 9 September 2026 30

General Introduction

A dashboard can present a convincing picture while still relying on an incorrect calculation. In insurance, a policy may have several coverages, several premium transactions and several claims. Joining all those records without first considering their level of detail can multiply amounts. Similarly, an extraction that misses an update can leave a dashboard out of date even when every subsequent task completes successfully.

This internship addressed the preparation work behind reliable reporting. Its central question was: *how can operational insurance records be turned into coherent, refreshable analytical datasets within a reproducible local environment?* The response was a complete prototype connecting an Oracle source to Tableau through incremental ingestion, object storage, Spark transformations and Airflow orchestration.

The source data were generated specifically for the project. No real customer dataset was used, because customer confidentiality had to be preserved. This made it possible to demonstrate business relationships and changes over time while keeping the work separate from the company's production systems.

The report follows four stages. chapter 1 introduces Sanlam Maroc, its wider group context and the internship assignment. chapter 2 reviews the concepts and alternative approaches that inform the solution. chapter 3 translates the requirements into an architecture, data model and transformation rules. chapter 4 describes the implementation, examines the available evidence and identifies the improvements needed beyond the demonstration environment.

Chapter 1

Company Background and Project Context

1.1 Sanlam Maroc at a glance

1.1.1 The host company

Sanlam Maroc is an insurer serving individuals, professionals and businesses in Morocco. Its activities include motor, home, health and life insurance, alongside protection and savings solutions. The company traces its origins to 1949 and adopted the Sanlam Maroc name in 2022, following Sanlam's acquisition of Saham Finances in 2018[1].

table 1.1 summarises selected figures from Sanlam Maroc's 2025 reporting. They describe the host company and are unrelated to the synthetic customers, policies and amounts used later in the technical project[2].

Table 1.1: Selected Sanlam Maroc company figures from 2025 reporting

Indicator	Reported figure
Insured customers	4.5 million
Employees	More than 824
Revenue in 2025	MAD 6.195 billion
Selected insurance activities	Motor, home, health, life and business insurance

Scale matters to the context of this project. A large insurance portfolio generates records throughout the customer relationship, from policy creation and renewal to premium collection and claims handling. An analytical platform must connect those events while keeping their meanings distinct. A premium written into a contract, for example, is not the same measure as a payment actually collected.

1.1.2 Products, distribution and the data they generate

Sanlam Maroc's published profile describes life and non-life activities supported by a network of more than 500 agencies. Its business lines include motor and motorcycle cover, health and protection, property, liability, leisure, savings and retirement[3]. These activities create several complementary views of the same customer: the person or business insured, the contracts held, the amounts due and the events that lead to claims.

For the prototype, that business context was reduced to a manageable set of entities. Customers and agents describe the parties involved. Products and policies describe the cover offered and purchased. Premium transactions describe collection activity, while claims and claim payments describe the handling of insured events. This simplified model gives the project a recognisable insurance structure without claiming to reproduce Sanlam's internal databases.

1.2 History and group context

1.2.1 Selected milestones

The company's history includes its establishment in 1949, its development under the Saham name and its subsequent integration into the Sanlam group. The 2018 acquisition and 2022 change of name explain the link between the Moroccan insurer and the Sanlam brand[1]. The timeline in figure 1.1 presents selected milestones as a sequence, rather than as a complete corporate history.

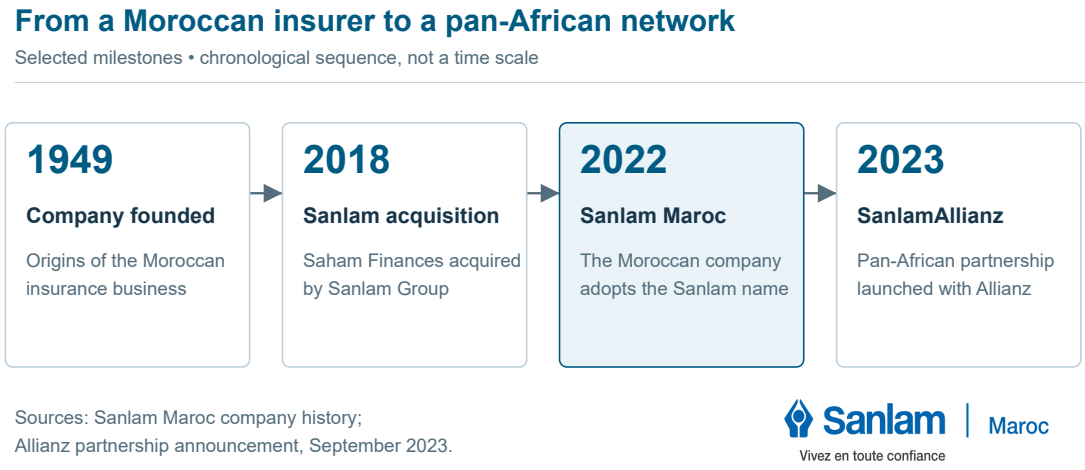


Figure 1.1: Selected milestones in the company's development. The milestones are shown in chronological order, with spacing that is not proportional to elapsed time. Sources: Sanlam Maroc and Allianz.

SanlamAllianz was launched in 2023 as a partnership bringing together the two groups' insurance and financial-services activities across African markets outside South Africa[4]. Sanlam Maroc belongs to this wider structure. In its 2025 annual-results announcement, Sanlam described the partnership as a 51/49 arrangement between Sanlam and Allianz[6]. The distinction between these entities is important: Sanlam Maroc is the host company, SanlamAllianz provides the pan-African context, and Sanlam Group has a broader set of businesses and investments.

1.2.2 A pan-African footprint within a wider group

figure 1.2 shows the 25 markets listed on the SanlamAllianz website consulted on 11 September 2026. Morocco is highlighted as the internship's country context. The world overview also identifies South Africa, India, the United Kingdom and Ireland as selected wider-group locations documented by Sanlam; it is not intended to inventory every subsidiary or investment[5, 6].



Figure 1.2: SanlamAllianz markets and selected wider Sanlam Group locations, based on sources consulted on 11 September 2026. Colour indicates presence and organisational scope, not revenue, market share or customer density. Map data: Natural Earth[7].

The 25 listed SanlamAllianz markets are Angola, Benin, Botswana, Burkina Faso, Burundi, Cameroon, Côte d'Ivoire, Egypt, Gabon, Ghana, Kenya, Madagascar, Malawi, Mali, Mauritius, Morocco, Mozambique, Namibia, Nigeria, Rwanda, Senegal, Tanzania, Togo, Uganda and Zambia[5].

Published totals need a date and a clear scope. Sanlam Maroc's 2025 integrated report presents a 27-country footprint, whereas the current SanlamAllianz site lists 25. Sanlam's March 2026 results announcement discusses exits from Zimbabwe and Niger, as well as the Zambian general insurance business[3, 6]. The map therefore uses the explicit website country list rather than combining totals from different reporting contexts. Zambia remains on that list; exiting one line of business does not necessarily mean leaving the country entirely.

The international footprint also helps explain the relevance of common data definitions. Similar questions can arise across markets, yet products, source systems and business conventions may differ. Comparing results requires agreement on what constitutes an active policy, a collected premium or a paid claim. The internship explores this principle within one local prototype; it does not attempt cross-country reporting.

1.3 Internship setting and assignment

The introductory internship took place in Casablanca from 4 August to 2 September 2026 as part of the Information Systems Engineering and Big Data programme at the *École Nationale des Sciences Appliquées de Berrechid*. Academic supervision was provided by Pr FAKHAR and company supervision by Mr HAIDER Mohieddine.

The internship was hosted within the *Data Factory*, the entity in which the project and the

accompanying data-engineering work were carried out.

The assignment was to design and automate a prototype data platform for insurance analytics. Work covered the complete path from a relational source to dashboard-ready outputs: building demonstration data, capturing changes, storing extraction batches, validating records, calculating analytical measures and coordinating the workflow.

Development began in notebooks, where individual transformations and their results could be examined interactively. The working logic was then organised into reusable Python functions and connected through an Airflow DAG. This progression made the pipeline easier to run consistently and separated the exploration of a rule from its scheduled execution.

1.4 Enterprise data architecture presented during the internship

The enterprise architecture presented during the internship follows a layered data-platform model. Operational systems capture insurance transactions; an ingestion layer transfers incremental changes to AWS; an S3-based data lake retains and organises those changes; Spark prepares analytical datasets; Airflow coordinates the executable workflow; and Tableau and Power BI deliver the results to business users. This separation protects transactional workloads from analytical processing and gives each stage a precise operational responsibility[11, 14].

1.4.1 Operational systems and off-hours ingestion

Daily activity begins in operational applications used to register customers, policies, payments, claims and other business events. These applications rely on several database technologies, including Oracle and PostgreSQL. They are optimised for reliable transaction processing during the working day, while historical analysis is delegated to the data platform.

The daily batch window starts at 19:00, after agencies close and the load on operational systems decreases. The ingestion workflow reads the changes recorded since the preceding successful checkpoint and transfers them to the cloud landing area. Scheduling the workload outside peak business hours limits competition for database, network and application resources.

The three-day lookback is treated as a recovery window rather than as three independent full loads. Each run re-reads recent additions and modifications, then uses stable business keys and update timestamps to retain the latest version. This pattern captures late commits and allows a failed daily run to be recovered without leaving a gap. At larger scale, the same contract can be implemented with database change data capture, which reads committed changes from database logs and maintains an ongoing replication position[10].

1.4.2 Cloud landing, data lake and processing

After extraction, the ingestion layer lands data in Amazon S3, which provides the durable object-storage foundation of the data lake. The lake is organised by source, table and processing date, and data advances through zones according to its readiness for consumption. AWS reference architectures separate ingestion, storage, cataloguing, processing, consumption, and security and governance; this division provides a professional operating model for the architecture discussed during the internship[11].

Hadoop-compatible interfaces provide distributed data access, while Amazon S3 remains the storage layer. Spark reads and writes S3 objects through the Hadoop connector, so the platform gains distributed processing without requiring the lake itself to be stored in HDFS[16]. A catalogue records dataset locations, schemas and partitions, while access control, encryption, monitoring and audit mechanisms apply across the platform.

The platform moves data through progressively refined representations. The Raw zone preserves source-oriented batches for traceability and replay. The Silver zone standardises schemas, applies quality controls and consolidates the latest valid record for each business key. The Gold zone publishes governed datasets with explicit grains, measures and dimensions for reporting. This zoned organisation is consistent with AWS guidance for raw, transformed and curated data and with the medallion pattern used in modern analytical platforms[12, 8].

1.4.3 Spark, Airflow and business intelligence

Spark performs the computational work over the retained data: it reads files, standardises fields, deduplicates records, joins business entities and calculates aggregates. PySpark provides the Python interface for these distributed transformations. Airflow orchestrates the complete executable chain by scheduling ingestion, launching processing jobs, enforcing dependencies, applying retries and recording task states. Spark therefore processes the data, while Airflow controls when and in what order the processing stages run[13, 14].

The final datasets are exposed to business-intelligence tools such as Tableau and Power BI. These tools consume prepared analytical data and present indicators to business users. They belong at the end of the flow rather than at the operational source: the data-engineering platform absorbs the work of extraction, consolidation and calculation before the dashboard is refreshed.

The architecture therefore follows a clear operating chain: operational systems record business events; the 19:00 incremental batch captures a three-day recovery window; Amazon S3 retains governed Raw, Silver and Gold datasets; Spark performs the transformations; Airflow controls the workflow; and BI tools present curated results. The local prototype reproduces these responsibilities on a smaller scale with one Oracle source, MinIO as S3-compatible storage, Spark transformations, an Airflow DAG and Tableau exports. It implements the same engineering principles without reproducing the scale or every managed service of the enterprise platform.

1.5 Problem statement, objectives and deliverables

1.5.1 The problem to solve

Moving data from one system to another is only part of the task. The pipeline also needs to decide which rows have changed, which version of a record should be retained and which relationships must hold before results are published. Once records have been consolidated, each measure needs an explicit grain so that joins do not inflate counts or amounts.

Automation introduces a further requirement: the stages must run in the correct order. Gold calculations depend on valid Silver tables, and the export depends on complete Gold outputs. A useful design must make these dependencies visible and stop downstream work when required inputs fail validation.

1.5.2 Objectives

The project was organised around five objectives:

1. Capture new and modified rows from nine source tables using a stored timestamp for each table.
2. Organise the local data lake into Raw, Silver and Gold layers using Parquet.
3. Check keys, relationships, types and selected business rules before producing analytical outputs.

4. Coordinate execution with Airflow and document the Docker environment needed to run it.

5. Export five stable CSV datasets for analysis in Tableau Desktop.

The deliverables include the synthetic source model and generator, four notebooks, reusable Python modules, the Airflow DAG, container configuration, analytical exports, a Tableau workbook and technical documentation. The five Gold datasets are not five dashboards: the saved workbook contains four dashboards, while policy status history is available as an additional export for future analysis.

1.6 Data confidentiality and project boundaries

1.6.1 Why the dataset is synthetic

The project does not use a real Sanlam customer dataset. Synthetic records were generated to preserve customer confidentiality while exercising realistic relationships such as policy ownership, premium collection and claim settlement. This is artificial data generation, not the anonymisation of a production extract.

Names, locations, amounts, risk scores and fraud flags are consequently demonstration attributes. The generator includes geographic conventions that do not represent the Moroccan portfolio. It also does not define a currency, so its amounts must not be presented as Sanlam results in Moroccan dirhams. Risk scores and fraud flags are simulated fields; no predictive model was trained as part of this project.

1.6.2 What the prototype represents

The project implements an Oracle source, MinIO object storage, Spark transformations, Airflow orchestration and Tableau visualisation. It uses S3-compatible interfaces without deploying an AWS environment, and Hadoop libraries without a separate Hadoop cluster. The early architecture discussions provided a conceptual starting point, not a verified map of Sanlam's production infrastructure.

Within that boundary, the aim was to build a complete and understandable demonstration. The work progressed from ingestion and metadata to object storage, transformation, reporting and automation. Validation at each stage helped stabilise the output schemas and clarify the responsibilities of the components before they were assembled into a scheduled workflow.

Chapter 2

State of the Art and Technical Foundations

The prototype combines several established ideas in data engineering. This chapter reviews the alternatives behind those choices: separating operational and analytical workloads, organising storage, capturing changes, processing records and coordinating execution. The purpose is to explain why the selected approach fits the internship, rather than to treat the chosen tools as the only possible solution.

2.1 From operational records to analytical datasets

2.1.1 Transactional and analytical workloads

Operational systems record individual business actions, such as issuing a policy, updating customer details or approving a payment. These transactional workloads, often called OLTP, prioritise consistent updates and precise access to records. A relational model supports them by defining keys and constraints between business entities.

Analytical workloads, commonly associated with OLAP, ask questions across many records: how a portfolio changes, which products account for most policies or how claim amounts vary over time. They rely on scans, joins and aggregations. Preparing a separate analytical representation allows those operations to be organised for reporting while controlling when and how the source is read.

This separation does not make an indicator correct by itself. Every measure still needs a population, a period and a unit. Written premium, collected premium, claimed amount and claim payment are different concepts. A reporting model should preserve those distinctions instead of presenting similar-looking amounts as interchangeable.

2.1.2 Grain and analytical modelling

The grain specifies what one row represents. It might be one customer, one policy, one claim or one month of premium due dates. Defining that grain before joining tables is essential because different source entities have different levels of detail.

A dimensional warehouse often separates measurable events into fact tables and descriptive attributes into dimensions. Another option is to publish enriched analytical tables tailored to particular reporting needs. The prototype follows the latter approach: Gold contains datasets with explicit grains, ready for Tableau. It does not implement a complete dimensional warehouse with historically versioned dimensions.

This choice keeps the demonstration manageable and reduces preparation inside the workbook. It also places responsibility on the transformation layer to aggregate related records correctly. Multiple one-to-many joins are a particular concern: joining coverages and premium transactions directly to a policy can repeat both sets of measures.

2.2 Analytical storage architectures

2.2.1 Warehouse, data lake and lakehouse

A data warehouse organises information into structures designed for analysis, usually with shared definitions and controlled schemas. A data lake stores files or objects at different stages of preparation. Its flexibility is useful for retaining original batches and supporting reprocessing, but the project must establish conventions for paths, schemas, quality and access.

A lakehouse adds managed-table capabilities to lake-style storage, including transactional behaviour and controlled data evolution. Simply placing Parquet files in three directories does not provide those guarantees. The implemented MinIO platform is therefore described as a data lake, not as a transactional lakehouse.

Table 2.1: Storage approaches considered for the internship

Approach	Main benefit	Design responsibility or trade-off
Relational warehouse	Shared structures for queries and measures	Reporting structures need to be designed in advance
File-based data lake	Retains batches and separates storage from processing	Schemas and consistency must be managed explicitly
Lakehouse	Managed tables over object storage	Additional transactional components and operational work

2.2.2 Progressive quality layers

The medallion pattern separates data into progressively refined layers, commonly called Bronze, Silver and Gold. Databricks describes these as ingestion, validation and business-oriented enrichment[8]. The project adopts the same separation of responsibilities but calls its landing layer Raw. It does not implement the transactional features of a managed lakehouse platform.

The three layers answer different questions: what arrived, what consistent state can be reconstructed, and what should be exposed for analysis? Keeping those representations separate makes checks and reprocessing easier to understand. The trade-off is additional storage and a longer sequence of transformations to operate.

2.2.3 Object storage and file formats

Object storage identifies a resource through a bucket and an object key. Prefixes within the key provide a logical organisation by layer, source table and ingestion date. The interface and the file format have different jobs: the S3-compatible API transfers objects, while Parquet describes the data inside them.

Parquet is a column-oriented format with compression and encoding support[9]. It is useful when analytical processing reads selected columns from structured datasets. CSV provides a simple exchange format, but does not preserve the same richness of type information. The prototype consequently uses Parquet inside the lake and CSV at the Tableau boundary. No benchmark was conducted to quantify a performance advantage for these choices.

2.3 Loading data and capturing changes

2.3.1 ETL and ELT

ETL transforms data before loading it into the analytical destination. ELT loads first and performs the main transformations within the target platform. The prototype is closer to ELT:

source batches are written to Raw before Spark builds Silver and Gold. Some technical work, including serialisation, still takes place during extraction.

Retaining the incoming batches makes it possible to revisit transformation rules without repeatedly querying the operational source. It also separates failures in extraction from failures in validation or aggregation, which helps locate problems in a multi-stage pipeline.

2.3.2 Batch and continuous processing

Batch processing groups work into identifiable runs. It suits periodic reporting and makes the inputs, outcomes and failures of an execution relatively straightforward to inspect. Continuous processing reduces the delay between an event and its use, but introduces additional concerns such as event ordering, late arrivals and processing state.

The internship did not require real-time reporting. A batch workflow was therefore appropriate to the scope and available environment. Its four-day schedule is a demonstration setting, not a freshness commitment for Sanlam's operational reporting.

2.3.3 Full loads, watermarks and change data capture

A full load rereads the source population at every execution. This is simple, but the amount transferred grows with the source. A timestamp watermark narrows extraction to records whose modification timestamp exceeds a stored boundary. It depends on that timestamp being updated reliably whenever relevant data change.

Change data capture, or CDC, can instead read transaction logs to identify inserts, updates and deletes. AWS DMS documents this approach for ongoing replication[10]. CDC describes how changes are captured; it is not synonymous with either a timestamp filter or continuous consumption.

Table 2.2: Comparison of extraction strategies

Strategy	Strength	Limitation or prerequisite
Full load	Simple reconstruction from the current source	Repeated transfer of the full population
Timestamp watermark	Reads new and modified rows	Requires reliable timestamps; a simple date filter does not capture deletes
Log-based CDC	Captures modification events	Requires access to logs and replication configuration and monitoring

A watermark was chosen because it provides a clear, observable incremental mechanism for the demonstration. It does not provide exactly-once delivery. If an object is written but the watermark update fails, a retry can read the same rows again. A late-committed change carrying an older timestamp can also be missed. These scenarios inform both the consolidation rules and the remaining limitations.

2.4 Processing engines, orchestration and deployment

2.4.1 Pandas, SQL and Spark

Pandas supports in-memory manipulation when a dataset fits the available machine. SQL can express joins and aggregations directly in a database. Spark SQL and DataFrames provide structured interfaces for processing data from several storage formats[13]. Choosing Spark should not be taken as evidence that the demonstration volume requires distributed computing.

The prototype uses Pandas to handle extraction batches and Spark to express reusable transformations over the lake. A SQL-only or Pandas-only solution could have been considered at this scale. Spark was selected to practise a data engineering architecture that can be extended, not because a comparative performance study established it as the fastest option.

2.4.2 A scheduler versus an orchestrator

A time-based scheduler can launch a script. An orchestrator also records dependencies, states and retries. An Airflow DAG defines the logical ordering of tasks, while Airflow's execution components run and monitor them[14]. This allows independent extractions to run concurrently and a downstream transformation to wait for all required inputs.

Orchestration does not make a task idempotent. A retry executes its code again, including any side effects. This matters here because the demonstration workflow generates source changes before extraction. In production, business applications would create those changes and the synthetic generator would be removed from the scheduled data pipeline.

2.4.3 Containers and dependency compatibility

Docker Compose describes related services, networks and volumes in a common configuration[15]. It makes the environment easier to start and inspect, but reproducibility still depends on pinned dependencies and external settings.

Spark's S3A integration illustrates this requirement. The Hadoop-AWS and AWS SDK libraries must match the Hadoop stack used by Spark. Hadoop-AWS provides the bridge from Hadoop-compatible filesystem operations to S3 object storage[16]. Using that bridge does not require a standalone Hadoop cluster.

2.5 Quality, traceability and the selected approach

Data quality has several levels: schema conformity, required keys, uniqueness, referential integrity and plausible business values. A missing field is not automatically invalid. A policy may have no agent, and a rejected claim may have no approved amount. Rules need to distinguish legitimate absence from a broken record.

Traceability links an output to its inputs and transformations. Retaining Raw batches is a starting point, but does not replace a run manifest or a catalogue. Likewise, a successful Airflow task confirms completion of the programmed work, not the business meaning of every resulting measure.

The selected design combines timestamp-based ingestion, a layered MinIO lake, Spark transformations and Airflow orchestration. It prioritises an understandable end-to-end workflow that can run locally. Recovery guarantees, memory limits and consistent publication across multiple tables remain explicit areas for improvement rather than assumed properties of the architecture.

Chapter 3

Requirements and Platform Design

The design translates the internship objectives into components, data structures and testable rules. It separates the operational model from the lake's processing layers and from the datasets exposed to reporting. Each boundary has a specific purpose: preserving inputs, establishing a consistent state or publishing business measures.

3.1 Requirements analysis

3.1.1 Users and analytical needs

Three functional roles guide the prototype. A developer configures and changes the pipeline. An operator monitors runs and investigates failures. An analyst interprets the outputs and builds reports. These are design roles, not a claimed organisational chart of Sanlam; during an introductory internship, one person may fulfil several of them.

The main analytical needs are to understand the customer portfolio, examine policies and their statuses, analyse claims and payments, and follow premium collection. Policy status history provides an additional event-level view of contract changes.

Table 3.1: Requirements and corresponding checks

ID	Requirement	Verification criterion
F1	Extract changes from nine source tables	A per-table timestamp filter; a Raw batch when rows exist
F2	Reconstruct a consistent state	Unique Silver keys and valid relationships
F3	Prepare analytical datasets	Five Gold outputs with defined grains
F4	Enforce execution dependencies	Silver after all ingestions; export after Gold
F5	Publish stable reporting files	Named columns in a consistent CSV order
Q1	Make execution observable	Task states and logs available in Airflow
Q2	Document the environment	Compose configuration, dependencies and example variables
Q3	Protect customer confidentiality	Synthetic records and separately managed credentials

The project defines no numerical throughput or response-time target. Its acceptance criteria concern coherence and end-to-end operation. Production capacity and availability would require a separate measurement campaign.

3.2 System architecture

Oracle supplies the source records. Python extracts changed rows and uploads their Parquet representation through Boto3. MinIO stores the Raw, Silver and Gold layers. Spark performs the transformations, and an export task produces CSV files for Tableau Desktop. Airflow controls when the executable stages run, while Docker provides their service environment.

The architecture in figure 3.1 separates storage from computation. MinIO is a separate container, not a service inside the Airflow Compose file. Python and Spark jobs execute in the

Airflow worker. Oracle and Tableau remain external to that task runtime. The presence of a Docker boundary therefore does not imply that every tool shown is packaged in one image.

The prototype deliberately preserves the division of responsibilities introduced in section 1.4, while reducing the number of systems and replacing cloud services with components that can run locally. table 3.2 makes this relationship explicit.

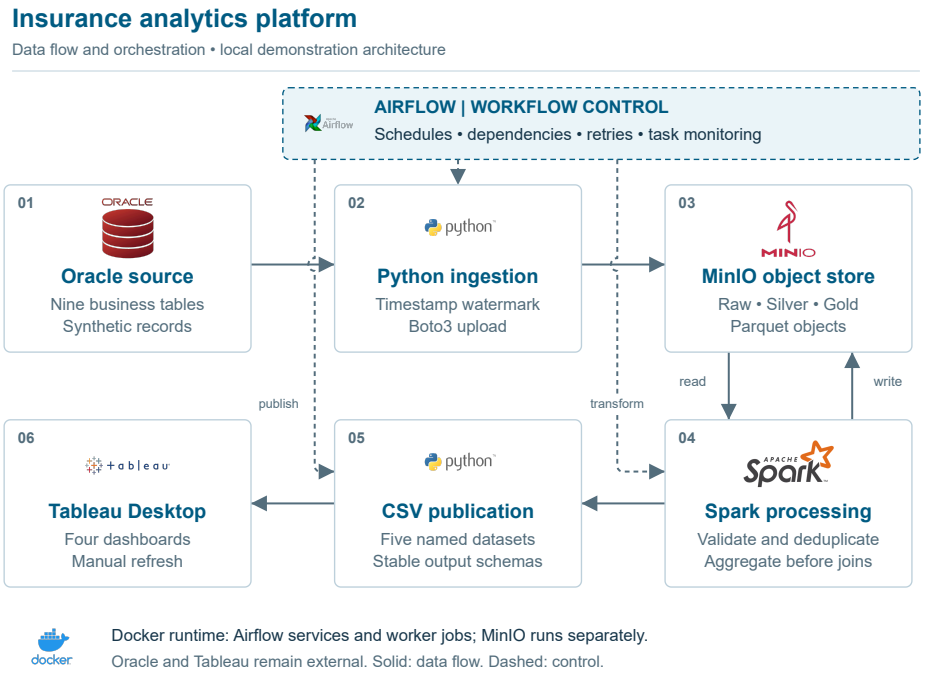


Figure 3.1: Implemented data platform. Airflow coordinates ingestion, Spark transformation and CSV publication. Solid arrows show data movement; dashed arrows show orchestration dependencies.

Table 3.2: Relationship between the enterprise concept and the implemented prototype

Enterprise concept discussed during the internship	Implementation in this project
Several operational source systems, including Oracle and PostgreSQL	One synthetic insurance schema in Oracle
Scheduled extraction after operational activity decreases	Airflow schedule used for demonstration, with per-table ingestion tasks
Incremental transfer of recent changes	A stored `LAST_UPDATED` watermark for each source table
AWS cloud and object-storage data lake	Local S3-compatible MinIO bucket
Distributed preparation with Spark	PySpark transformations in the local and container runtime
Workflow dependencies and monitoring	Airflow DAG, task states, logs and retries
Tableau and Power BI consumption	Five CSV exports feeding a Tableau workbook
Enterprise governance and operations	Project-level validation and documentation; broader controls remain outside scope

3.3 Source model and lake layers

3.3.1 Nine business entities and one control table

The source model contains nine related business tables. Their keys and relationships are shown in figure 3.2.

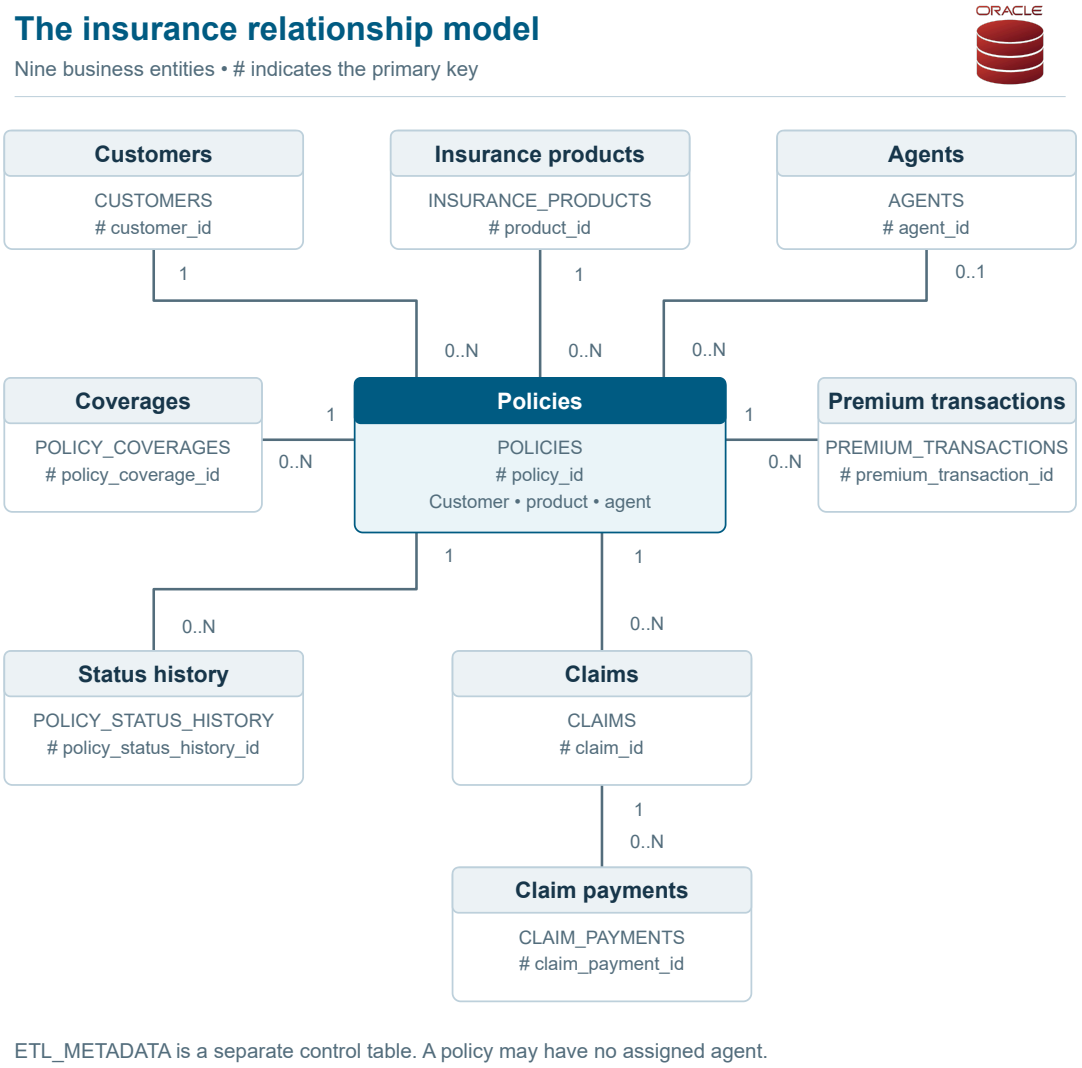


Figure 3.2: The nine-entity demonstration model. Cardinalities describe permitted relationships, and the relationship between a policy and an agent is optional.

`CUSTOMERS`
: Customer identity, demographics, segmentation and simulated risk attributes.

`AGENTS`
: Agent identity, code, region and activity status.

`INSURANCE_PRODUCTS`
: Product names, lines of business, premium assumptions and coverage limits.

`POLICIES`
: Contracts linking customers, products and optional agents, with dates, status, premium and insured amounts.

`POLICY_COVERAGES`
: Coverage limits, deductibles and activity flags associated with a policy.

`POLICY_STATUS_HISTORY`
: Policy status changes, reasons and change timestamps.

`PREMIUM_TRANSACTIONS`
: Due dates, transaction dates, amounts, transaction types and collection statuses.

`CLAIMS`
: Claim type, dates, status, amounts, handler and simulated fraud flag.

`CLAIM_PAYMENTS`
: Payments attached to claims, with dates, amounts and payment statuses.

`ETL_METADATA` is a separate control table. It holds one watermark per source table and is not part of the insurance relationship model. A claim can be traced through its policy to the corresponding customer and product; its payments remain a separate one-to-many collection.

3.3.2 Raw: preserve the extracted batches

Raw is the landing area for source-shaped Parquet batches. Object keys identify the table, ingestion date and individual batch:

```
raw/<TABLE_NAME>/load_date=YYYY-MM-DD/<TABLE_NAME>_<UTC_TIMESTAMP>.parquet
```

The `load_date` partition describes when the pipeline extracted the batch in UTC. It is not a business date from Oracle. Loss dates, reporting dates and policy inception dates remain separate fields inside the records.

3.3.3 Silver: reconstruct and validate the current state

Silver provides a consistent analytical base. Spark standardises names, casts dates and decimal amounts, resolves repeated versions and applies business and relationship checks. The source filename is used temporarily to order versions, then removed before publication. The ingestion date remains available.

3.3.4 Gold: organise data for analysis

Gold reshapes validated records around reporting needs. The five outputs cover customers, policies, claims, premium collection and status history. The transition between these layers is illustrated in figure 3.3; the layers represent different responsibilities rather than different storage products.

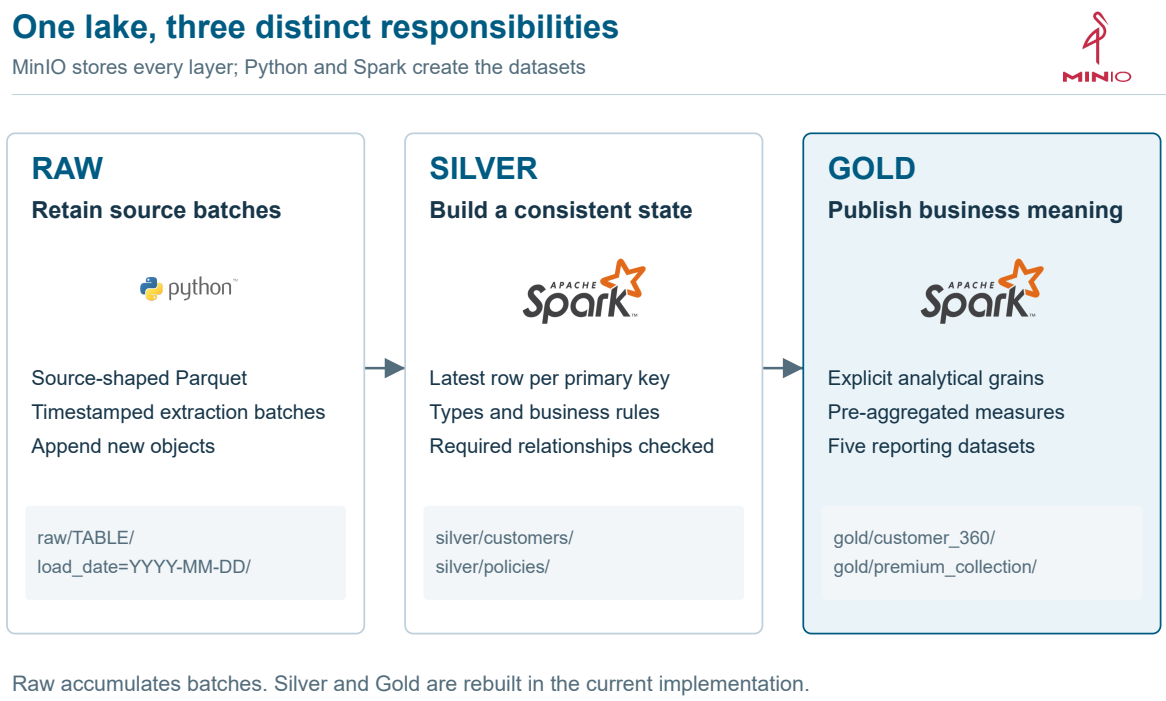


Figure 3.3: The lake's three quality layers: retained source batches, validated current records and business-oriented outputs. All three are stored in MinIO.

3.4 Incremental ingestion design

3.4.1 The watermark contract

For a source table T, `ETL_METADATA` stores the most recently committed watermark w_T. Initialising this value to an old timestamp allows the first extraction to read the existing population. Each later batch is defined by:

$$B_T = \{r \in T \mid r.\texttt{LAST_UPDATED} > w_T\}.$$

An empty batch creates no object. For a non-empty batch, the pipeline writes the Raw object before committing the maximum extracted timestamp as the new watermark. The comparison is strictly greater than; the implementation has no overlap window or delete capture.

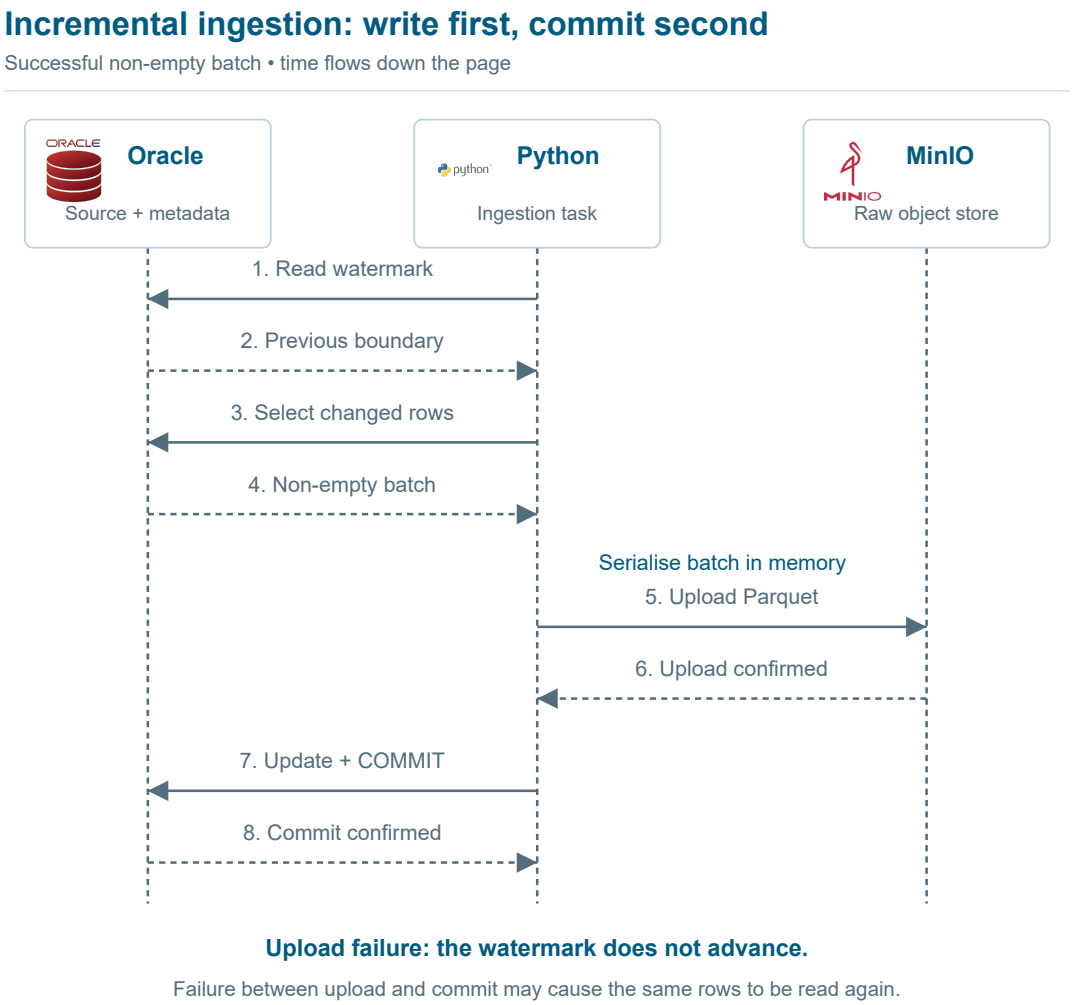


Figure 3.4: A successful, non-empty ingestion. The Raw upload completes before Oracle commits the new watermark. Dashed message arrows are responses, not orchestration links.

3.4.2 Failure and recovery boundaries

Writing the object before advancing the watermark prevents an upload failure from being recorded as a completed extraction. There is nevertheless a gap between the two operations. If the object exists but the metadata commit fails, a retry can read the same records again. Silver's consolidation handles repeated versions, but it does not create a distributed transaction between Oracle and MinIO.

Tables are extracted independently, without one shared Oracle snapshot. If the source changes between extractions, Silver may encounter a relationship whose referenced row has not yet arrived. The checks should stop publication in this situation. Handling late commits, deletions and cross-table consistency more comprehensively is part of the production evolution, not an existing guarantee.

3.5 Analytical output design

3.5.1 One grain for each Gold dataset

Table 3.3: Gold datasets and their intended grains

Dataset	One row represents	Analytical use
`customer_360`	One customer	Policy, claim and payment summary
`policy_performance`	One policy	Product, customer and agent context with related aggregates
`claims_performance`	One claim	Claim status, amounts and payments
`premium_collection`	One due month	Collection amounts, counts and rates
`policy_status_history`	One status event	Policy changes over time

Column names and grain form a contract between preparation and visualisation. A missing required field should fail the export check, and duplicate analytical keys should be detected before the affected dataset is published.

3.5.2 Preventing double counting

Suppose a policy has two coverages and three premium transactions. Joining those collections directly produces six combinations. Summing transaction amounts afterwards would count each transaction twice. Adding claims to the same join could multiply the result again.

The design therefore aggregates each collection to the required grain before joining it to the policy or customer. Paid claim payments are first grouped by claim and then, where needed, by policy. Premium transactions are aggregated separately by status. This keeps the meaning of each measure tied to the level at which it was calculated.

3.5.3 Measure definitions

Paid claim amount includes only payment records whose status is `PAID`. In the customer and policy outputs, `total_paid_amount` refers to claim payments. Collected premium is represented by `paid_total_amount`. Similar column names must not obscure the fact that these are different flows of money.

For due month m, the demonstration's amount-based collection rate is:

$$R_m = \frac{A_{\text{PAID},m}}{A_{\text{PAID},m} + A_{\text{FAILED},m} + A_{\text{PENDING},m} + A_{\text{REVERSED},m}},$$

where $A_{s,m}$ is the sum of transaction amounts for status s and month m. The rate is defined when the denominator is positive. The count-based rate uses transaction counts instead of amounts. Exports store fractions; percentage formatting belongs to the reporting layer. This is an explicit demonstration convention, not an accounting measure net of refunds.

3.6 Quality and publication design

Silver checks required keys, accepted status values and foreign-key relationships. Business rules also recognise legitimate missing values: a customer may lack an email address, an initial status event may have no previous status, and a rejected claim may have no approved amount. Filling these fields indiscriminately could change their meaning.

Silver is reconstructed from Raw and written one table at a time. Gold is then calculated and published one dataset at a time. This is straightforward at demonstration scale, but it does not provide an atomic switch across the complete set of outputs. A shared production platform would benefit from versioned runs whose outputs become visible only after the whole version passes validation.

These decisions establish what the implementation must do and what it does not yet guarantee. The next chapter connects them to the code, runtime and evidence available in the project.

Chapter 4

Implementation, Validation and Evaluation

The implementation connects the notebook experiments to an automated execution path. This chapter explains the repository, the processing stages and the runtime, then examines the screenshots and exported data retained from development. The final sections distinguish the demonstrated results from the work still needed for a production deployment.

4.1 Repository organisation

The repository separates exploratory notebooks from the reusable code called by Airflow. This makes it possible to inspect how transformations were developed without requiring interactive notebook execution during a scheduled run.

```
SANLAM_PROJECT/
|-- README.md
|-- .env.example
|-- requirements.txt
|-- dashboard.twb
|-- src/
|   |-- __init__.py
|   |-- oracle_db_connection.py
|   |-- seed_insurance_oracle.py
|   |-- spark_minio.py
|   +-- pipeline_jobs.py
|-- notebooks/
|   |-- 01_ingest_oracle_to_raw.ipynb
|   |-- 02_transform_raw_to_silver_final.ipynb
|   |-- 03_transform_silver_to_gold_final.ipynb
|   +-- 04_export_gold_for_tableau.ipynb
|-- sql/SANLAM.sql
|-- airflow/
|   |-- Dockerfile
|   |-- docker-compose.yaml
|   |-- requirements-sanlam.txt
|   +-- dags/sanlam_insurance_pipeline.py
|-- tableau_exports/
|-- docs/
|   |-- architecture.md
|   |-- setup.md
|   |-- troubleshooting.md
|   |-- sanlam_project_report.tex
|   |-- report_en/
|   +-- figures/
+-- output/pdf/
```

4.1.1 Responsibilities of the main files

`pipeline_jobs.py`
Implements ingestion, Silver and Gold transformations, validation and export functions.

spark_minio.py
Configures Spark's S3A access, Hadoop-AWS dependencies and MinIO endpoint.

oracle_db_connection.py
Provides the shared Oracle connection helper.

seed_insurance_oracle.py
Creates the synthetic business schema and generates initial records and subsequent changes.

sql/SANLAM.sql
Initialises extraction metadata; the Python generator creates the business tables.

notebooks/
Records the interactive ingestion, transformation and validation workflow.

airflow/
Contains the image definition, service configuration and scheduled DAG, with directories for runtime configuration and plugins.

tableau_exports/ and dashboard.twb
Hold generated reporting files and the Tableau workbook that consumes them.

docs/
Explains architecture, setup, troubleshooting and the report itself.

Generated CSVs, Airflow logs and the local virtual environment are ignored by the repository configuration. Credentials are supplied separately through environment files; `.env.example` documents the required variable names without real secrets.

4.2 Implementing the processing stages

4.2.1 Oracle to Raw

The ingestion function first validates the requested table against the permitted source tables. It reads that table's watermark, selects rows with a later `LAST_UPDATED` value and serialises the result as an in-memory Parquet batch. Boto3 then uploads the object to MinIO through its S3-compatible API.

The order of operations is deliberate:

1. Read the previous watermark from `ETL_METADATA`.
2. Select new and modified rows from the chosen source table.
3. Serialise a non-empty batch to a memory buffer.
4. Upload its Parquet object to Raw.
5. Determine the maximum modification timestamp in the uploaded batch.
6. Commit that value as the table's new watermark in Oracle.

If there are no changed rows, the task returns without creating an object. When an upload fails, the watermark is not advanced. This preserves the opportunity to extract the outstanding rows again, subject to the recovery limitations discussed in chapter 3.

4.2.2 Raw to Silver

Spark reads the Parquet history under each Raw table prefix and temporarily adds the physical source filename. A shared configuration describes primary keys, date fields, decimal fields, allowed values and relationship checks. The operational module is the reference for the Airflow execution path; the notebooks explain how its transformation rules were developed.

For each table, a window partitions records by primary key and orders versions by modification timestamp, ingestion date and source filename, in descending order. The first record is retained. This provides a deterministic reconstruction rule for the available history; it cannot recover an update that was never captured in Raw.

Validation checks non-null and unique keys, accepted statuses and foreign-key resolution. Left anti joins identify child records without the required parent. Date rules include the ordering of loss and reporting dates and the 365-day duration used by the synthetic policy generator. Approved amounts must not exceed claimed amounts. Claims in `SETTLED` or `CLOSED` require a closure date, while permitted missing values are handled separately.

All Silver validations run before that layer's writes begin. Validated tables are then written successively in overwrite mode with Snappy compression. The temporary source-file field is dropped at publication, while the ingestion date is retained.

4.2.3 Silver to Gold

Gold joins the validated tables and calculates the five analytical datasets. Claims, claim payments, coverages and premium transactions are aggregated to the required grain before they are combined. Premium statuses are pivoted into named count and amount fields. Explicit aliases keep similarly named attributes unambiguous.

Each output is checked for duplicate analytical keys before it is written. The result is a collection of datasets whose rows have a defined interpretation, rather than one large join containing every source record. Publication remains sequential across outputs, so a successful write of one dataset does not imply that all five have been replaced atomically.

4.2.4 Gold to Tableau exports

The export stage reads Gold, checks that the required columns exist and selects them by name in the expected order. It also normalises the names created by the status pivot. These checks protect the reporting contract from accidental schema changes: a value must be associated with its semantic column, not simply its position.

One UTF-8 CSV is written for each Gold dataset in `tableau_exports/`, without a Pandas index. The conversion through `toPandas()` collects each output into memory before writing. That is practical for the demonstration volume, but limits the size of an individual export.

Airflow automates file creation. Tableau Desktop refreshes those local files manually. Publishing a hosted data source and scheduling its refresh is outside the implemented workflow.

4.3 Connecting Spark to MinIO

MinIO provides local object storage through an S3-compatible interface. Python ingestion accesses it directly using Boto3; Spark accesses it through Hadoop's S3A filesystem connector. Typical paths are:

```
s3a://sanlam-data-lake/raw/CLAIMS/
s3a://sanlam-data-lake/silver/claims/
s3a://sanlam-data-lake/gold/claims_performance/
```

S3A translates Spark's filesystem operations into object-storage requests. Hadoop-AWS and the AWS SDK are Java dependencies required by that integration. They do not represent a separately deployed Hadoop cluster, nor does the S3-compatible protocol imply that these objects are stored in an AWS account.

The Spark session sets its logging level to `WARN` to keep task logs readable while preserving warnings and errors. Each job stops the session when finished so that its JVM resources and connections are released.

4.4 Airflow orchestration

The DAG `sanlam_insurance_incremental_pipeline` contains thirteen tasks: one synthetic-change generation task, nine ingestion tasks, two Spark transformations and one export task. The ingestion tasks are declared explicitly for the nine tables; they do not use dynamic task mapping.

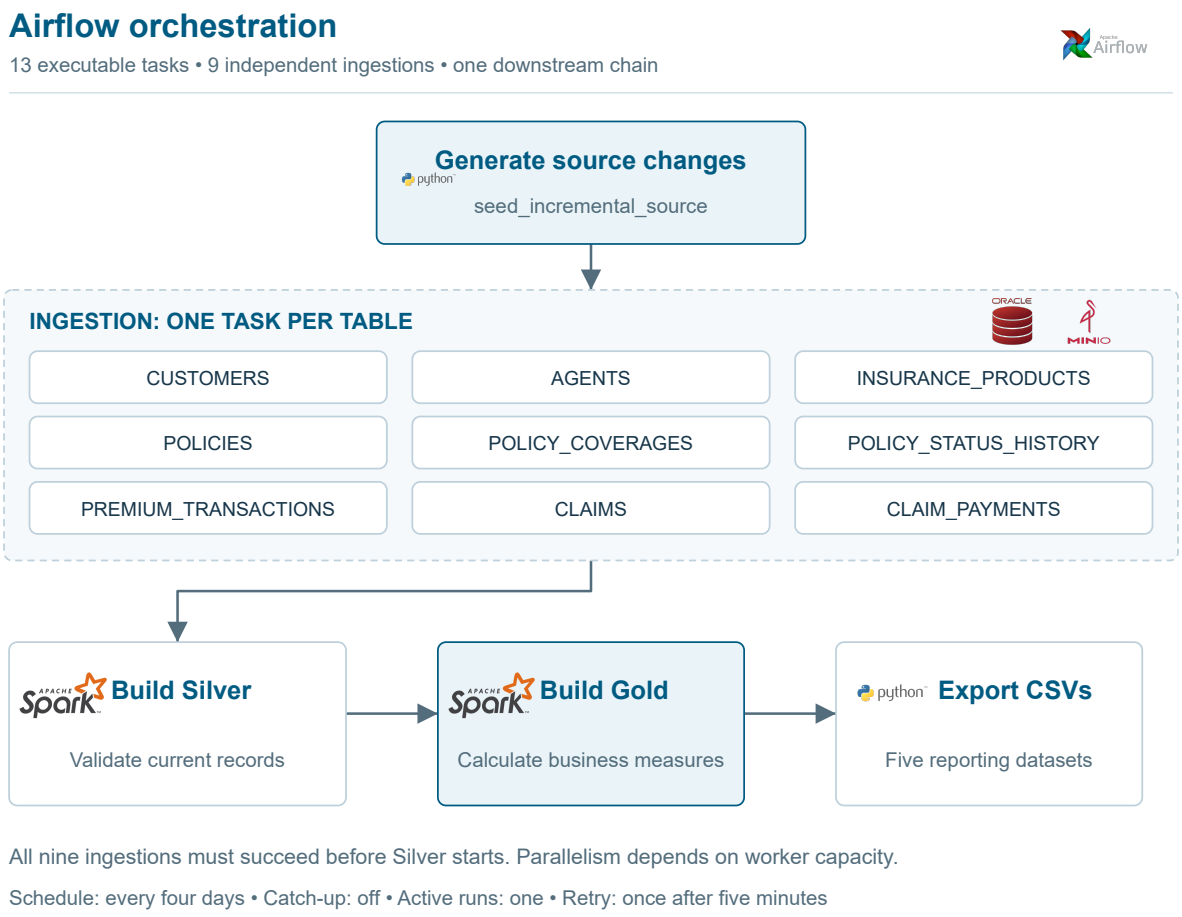


Figure 4.1: The thirteen-task Airflow workflow. Nine independent ingestions may run concurrently, subject to available worker capacity; Silver waits for all nine.

`seed_incremental_source`
: Generates controlled additions and updates in the synthetic source.

`ingest_*`
: Invokes the extraction function once for each business table.

`transform_raw_to_silver`
Reconstructs and validates Silver using Spark.

`transform_silver_to_gold`
Calculates the analytical outputs.

`export_gold_for_tableau`
Produces the five reporting CSVs.

The scheduler identifies tasks whose dependencies are satisfied. The worker imports the mounted project code and executes the functions. A failed upstream task prevents dependent work from proceeding until a retry or subsequent recovery succeeds.

The configured schedule is every four days, starting from 17 August 2026. Catch-up is disabled with `catchup=False`, and `max_active_runs=1` prevents overlapping DAG runs. A failed task has one retry after five minutes. These are demonstration settings that would need to be reconsidered against a production operating window.

The source-generation task creates related records as well as updates, so its input parameter is not an exact count of inserted rows. It also has side effects: rerunning it changes the source again. A production pipeline would remove this synthetic step and capture changes created by the actual business systems.

4.5 Docker runtime and reproducibility

Docker packages the execution environment, while Compose describes the related services. The Airflow stack includes an API server, scheduler, DAG processor, worker and triggerer. PostgreSQL stores Airflow metadata, and Redis supports worker messaging. The worker image supplies Python, Java, PySpark, Oracle client dependencies and the required S3A libraries.

Table 4.1: Versions defined for the containerised Airflow execution path

Component	Version	Responsibility
Apache Airflow	3.3.1	Scheduling and task monitoring
Java	17	Spark JVM runtime
PySpark	4.0.1	Data transformations
Hadoop-AWS	3.4.1	S3A connector
AWS Java SDK bundle	2.24.6	S3 requests used by the connector
PostgreSQL	16	Airflow metadata
Redis	7.2	Worker message queue

The Windows project directory is mounted at `/opt/sanlam` inside the containers. This lets the worker import `/opt/sanlam/src/pipeline_jobs.py` while development takes place in the host directory. Changes to mounted source files are normally visible without rebuilding the image; changes to the Dockerfile or installed dependencies require a rebuild and container recreation.

MinIO runs separately, exposing its API on port 9000 and console on port 9001. Containers reach host services through `host.docker.internal`. The `minio-data` volume preserves objects, and the PostgreSQL volume preserves Airflow metadata. Persistent data therefore have a different lifecycle from the containers that serve them.

The local virtual environment inspected during the earlier review contained a different PySpark version from the containerised stack. The notebooks remain useful for exploration, but reproducing the automated workflow should use the dependencies defined for the Airflow image. Aligning the two profiles would make maintenance more predictable.

4.6 Visual evidence and analytical outputs

4.6.1 Execution and storage evidence

The following screenshots were retained during development. They illustrate a completed Airflow run and the organisation of the MinIO lake. They are historical evidence of the implementation, rather than screenshots of a new live execution performed during this report revision.

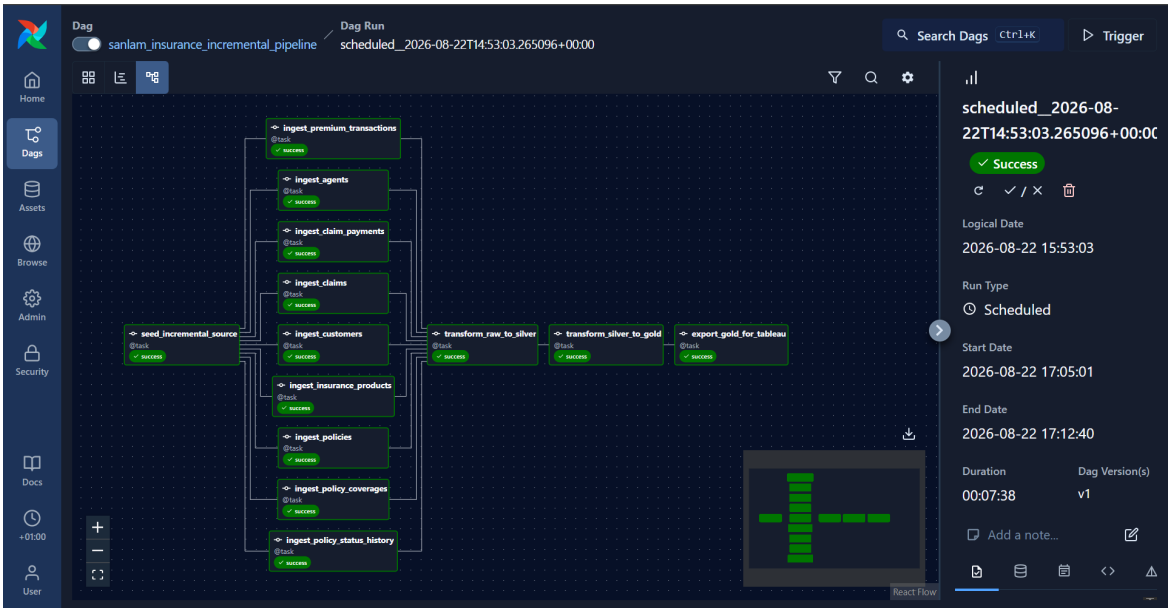


Figure 4.2: A completed Airflow run showing source generation, ingestion, Spark transformations and CSV export.

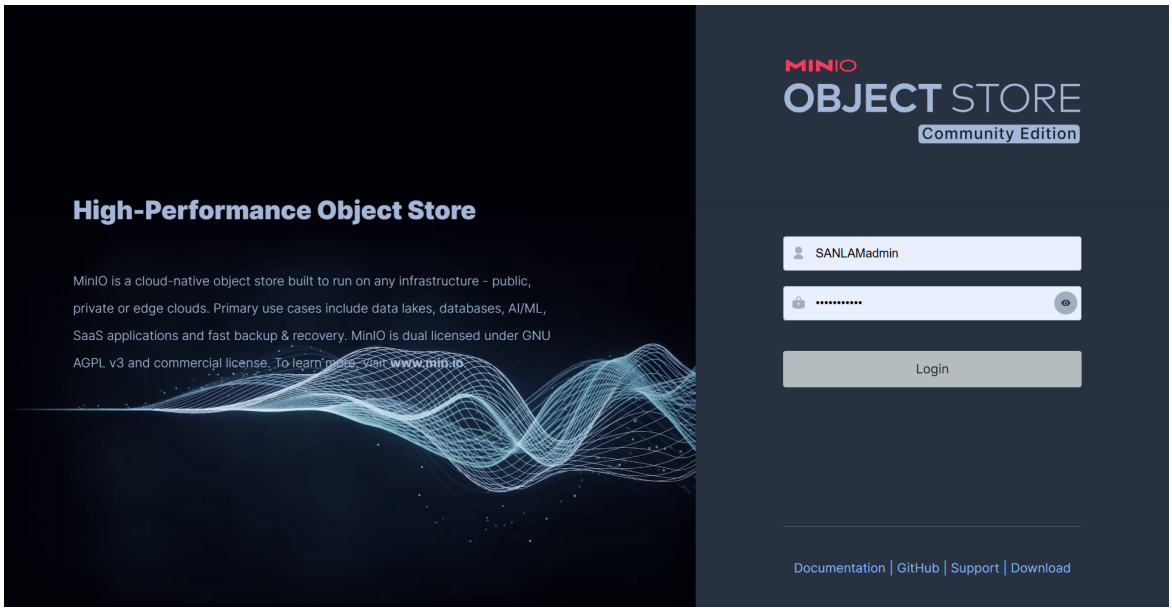


Figure 4.3: MinIO's sign-in interface for the local S3-compatible object store.

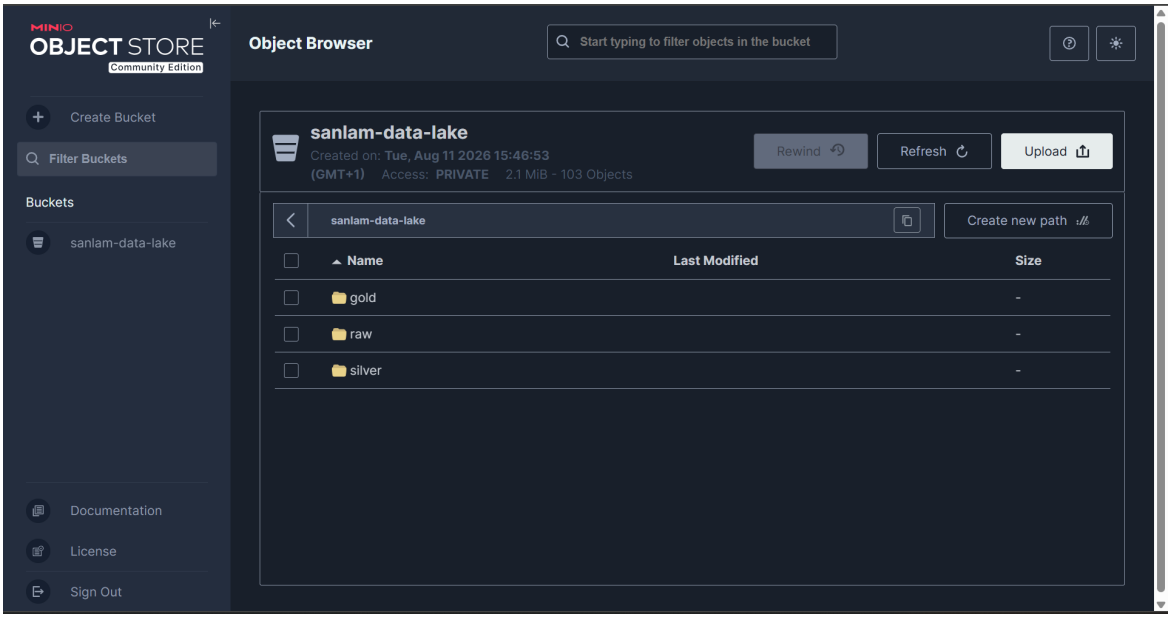


Figure 4.4: The `sanlam-data-lake` bucket and its Raw, Silver and Gold prefixes.

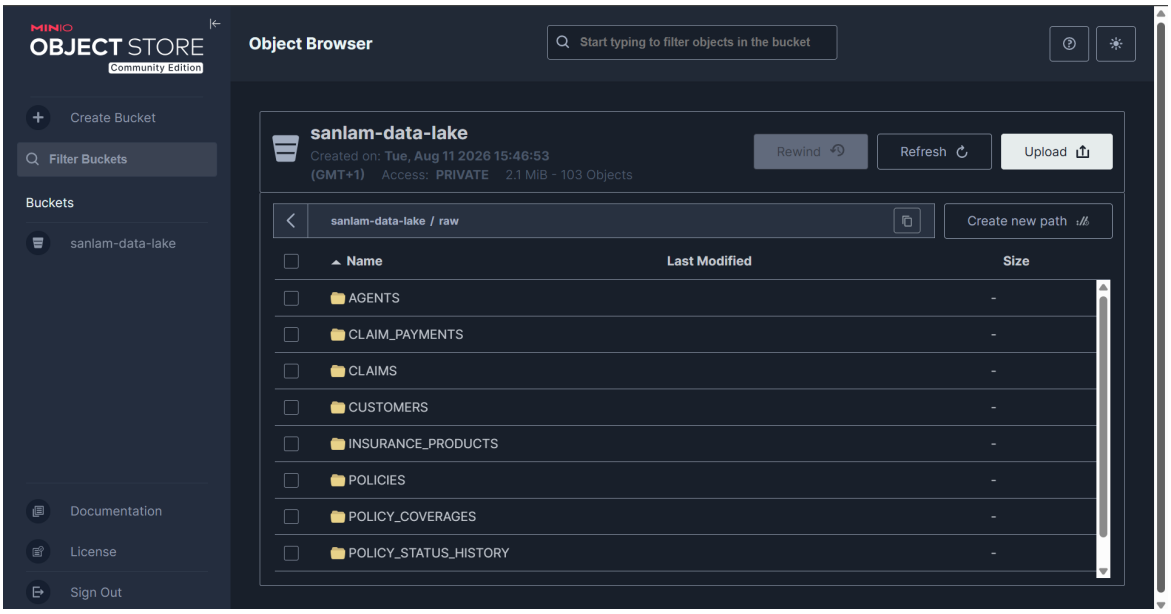


Figure 4.5: Raw prefixes corresponding to the nine Oracle business tables.

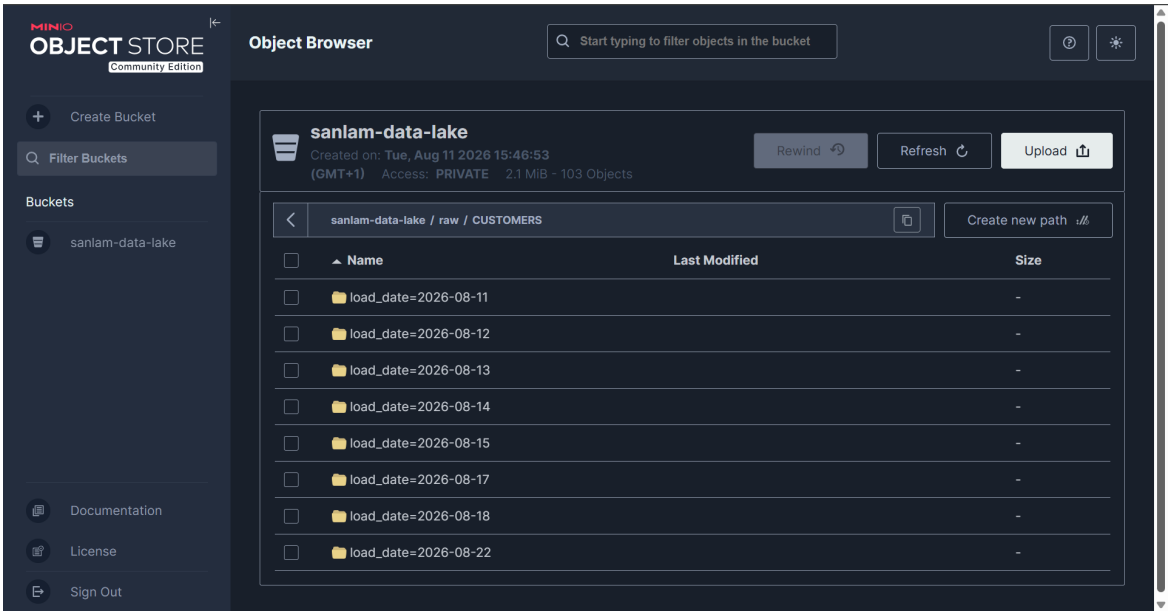


Figure 4.6: Customer ingestion batches organised by `load_date`. This date identifies extraction, not the business event.

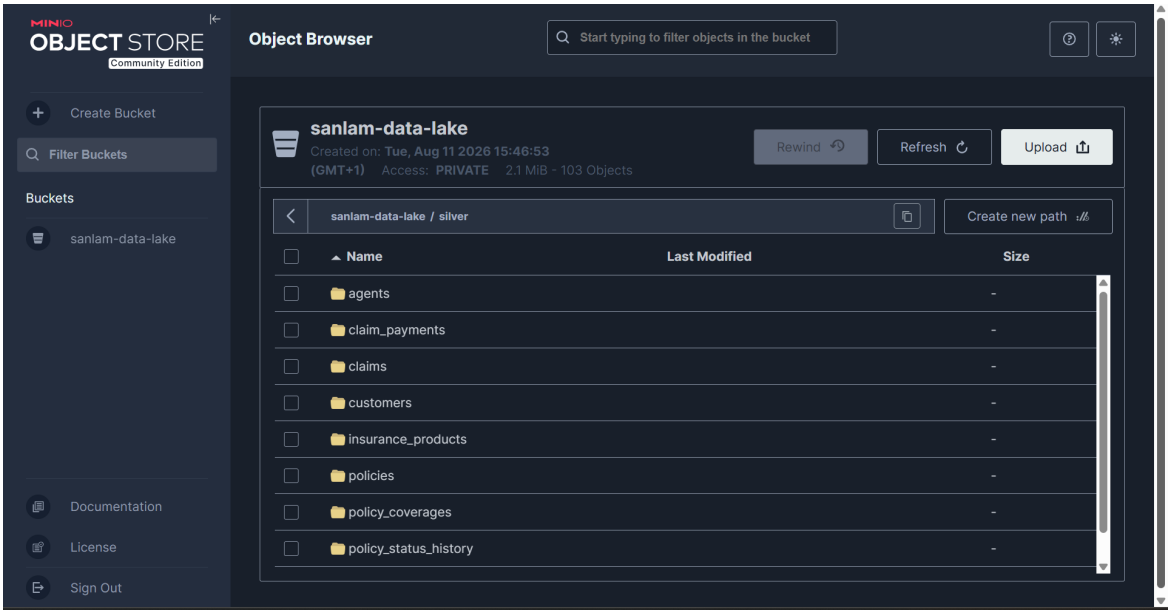


Figure 4.7: The Silver layer with standardised, lower-case dataset names.

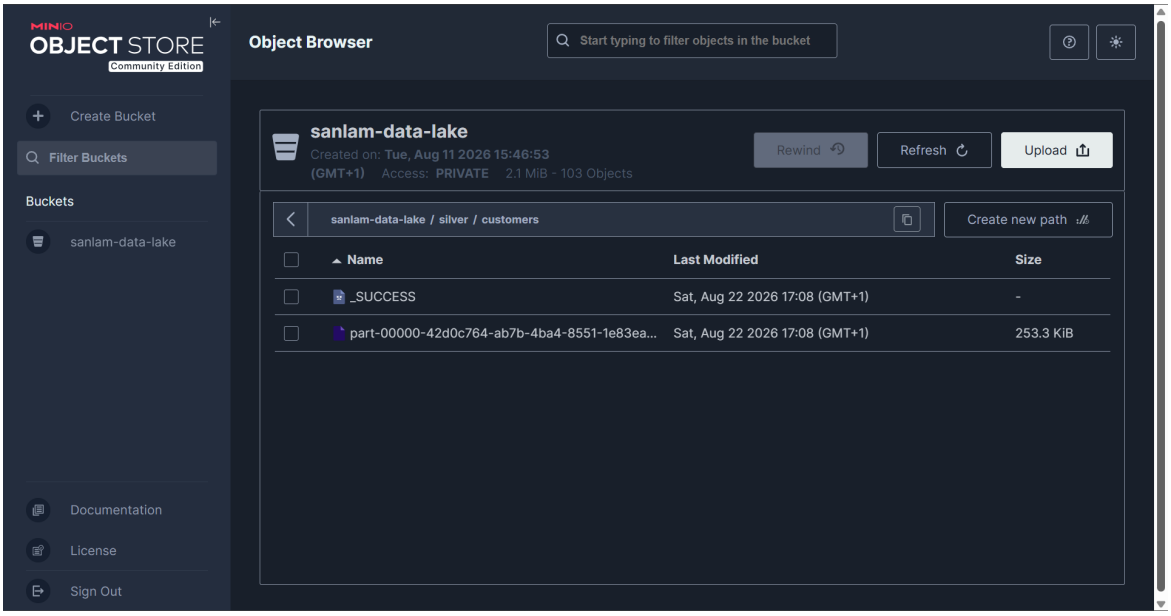


Figure 4.8: A published Silver customer dataset, including its Parquet file and Spark success marker.

4.6.2 Tableau dashboards

The Tableau views present the analytical outputs in a form that supports comparison and exploration. They group related measures instead of exposing the underlying source tables directly. Risk scores and fraud flags remain synthetic attributes; their appearance in a chart does not turn them into model predictions.

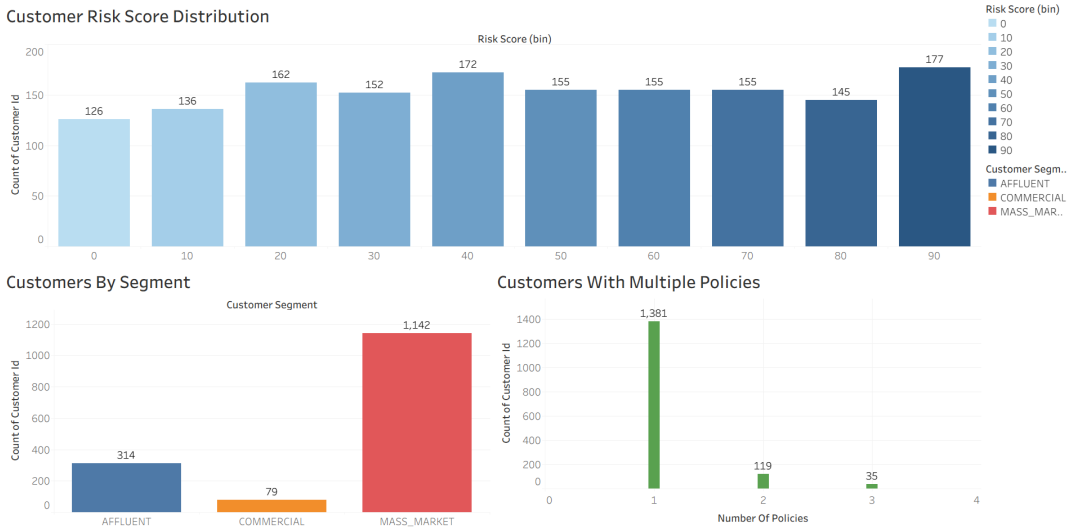


Figure 4.9: Customer 360 dashboard: simulated risk-score distribution, customer segments and customers holding multiple policies.

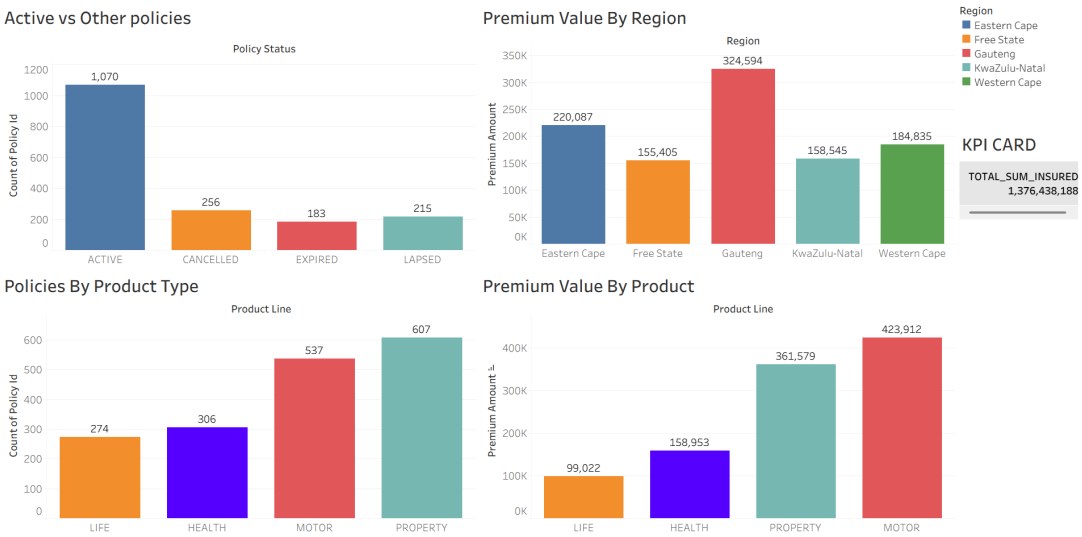


Figure 4.10: Policy dashboard: policy statuses, premiums by region, policy counts by product and premiums by product.

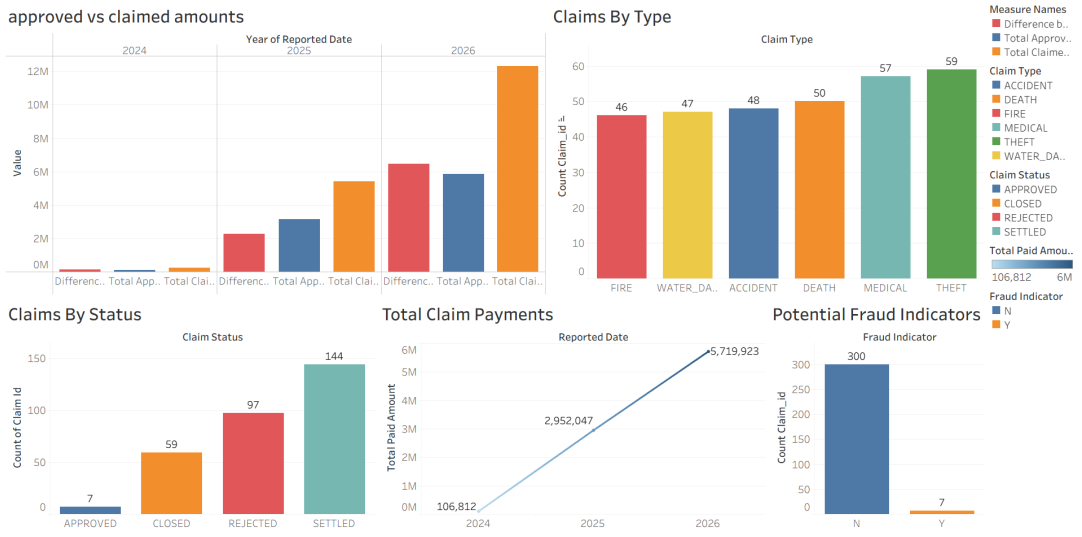


Figure 4.11: Claims dashboard: claimed and approved amounts, claim types and statuses, payment years and simulated fraud flags.

Five CSV files form the reporting interface. They cover customer profiles, policy performance, claims, premium collection and policy status history. table 3.3 identifies each dataset and defines its grain. Keeping these definitions outside the workbook helps prevent a visual change from silently changing the meaning of an indicator.

4.7 Validation results and their scope

4.7.1 Validation approach

Development checks examined row counts, Parquet and CSV schemas, key relationships and important totals. They also included inspection of Tableau views and successful completion of the Airflow chain. Technical checks and business interpretation serve different purposes: a valid schema can still contain a measure calculated at the wrong grain.

The notebooks and operational module contain table-level rules and relationship checks. The export additionally enforces required Gold columns. These checks can detect structural problems before CSV publication, although they are not yet exposed as a separate, persistent quality report for every run.

4.7.2 Verified export snapshot

The canonical CSV files dated 1 September 2026 were inspected during the documentation review on 9 September. Their dimensions are shown in table 4.2. Headers matched the expected columns, and the published grain keys were neither blank nor duplicated in that snapshot.

Table 4.2: Available synthetic CSV snapshot, inspected on 9 September 2026

File	Rows	Columns
`customer_360.csv`	2,075	16
`policy_performance.csv`	2,264	32
`claims_performance.csv`	371	24
`premium_collection.csv`	32	11
`policy_status_history.csv`	2,964	16

Cross-export checks reconciled 2,264 policies and 371 claims. Claimed amounts totalled 21,282,223.18, approved amounts 10,659,267.45 and paid claim amounts 10,362,444.57, in the generator's unspecified monetary unit. These figures test consistency between demonstration outputs; they are not financial results for Sanlam.

Syntax checks of six project Python files and twenty-four notebook code cells found no syntax errors. Such a check does not execute connections or transformations. The retained screenshots and development validations document earlier runs, while the report revision itself does not constitute a fresh end-to-end Oracle, MinIO and Airflow test.

4.7.3 Workbook state and remaining limitations

The saved Tableau workbook contains four dashboards and seventeen worksheets. Screenshots are available for the customer, policy and claims views. A premium-collection dashboard exists in the workbook, but no screenshot has been fabricated for it. Policy status history is exported without a dedicated connected dashboard.

Two workbook connections still reference files with the `_fresh` suffix, for policy performance and premium collection. They need to be redirected to the canonical exports and refreshed before the workbook can be treated as the same snapshot used in the checks above. The historical screenshots are therefore not evidence for those exact totals.

Ingestion is incremental, but Silver rereads Raw history and Gold is rebuilt. The whole pipeline is not incrementally updated at every stage. Sequential table writes and direct CSV writes also lack a single atomic publication boundary. In addition, some transformations reduce timestamps to dates, including policy status history, so intraday analysis would require preserving finer temporal detail.

These limits define the reach of the evidence. The project demonstrates a working data path, explicit transformation rules and coherent available outputs. It does not establish continuous availability, loss-free capture in every failure scenario or production-scale performance.

4.8 Security and operational improvements

4.8.1 Configuration and secrets

Credentials should remain in environment files or a secrets system, outside publishable source files. The example environment file documents the configuration contract. A shared deployment would benefit from Airflow connections or an enterprise secrets manager so that credentials can be rotated without modifying code.

Reproducibility depends on more than the Python packages. It also includes the image, Java libraries, service configuration, Oracle schema, synthetic generator, environment variables, output schemas and documented startup order. Keeping those elements aligned is necessary for another developer to reproduce the automated workflow.

4.8.2 Quality reporting and batch traceability

The existing checks could be represented as dedicated Airflow tasks, with results stored by run. This would make failures in uniqueness, required fields, accepted values, dates, amounts or relationships easier to see without inspecting transformation logs.

A batch manifest could record the source table, extraction time, old and new watermarks, row count, object key and outcome. This would connect an analytical result to an identifiable Raw batch. Monitoring watermark progression could also flag a table that stops advancing or produces an unexpectedly empty batch.

4.8.3 Recovery and consistent publication

The first reliability improvements should address late changes and deletes, then publish outputs as a validated version. A temporary area could hold the complete result of a run before a controlled switch exposes it to consumers. Reconciliation between stored objects and metadata would help recover from failures between upload and watermark commit.

Failure-injection tests and load measurements would then provide evidence for the resulting guarantees. They are more useful than assuming that retries alone make a multi-system workflow safe.

4.8.4 Governed reporting and maintainability

Hosted reporting could publish a governed data source through Tableau Cloud or Tableau Server, with an appropriate access and refresh mechanism. This is a future deployment choice rather than a requirement of the local internship prototype.

Continuous integration could check Python imports, DAG parsing, output contracts and small transformation cases before a scheduled run. As the code grows, `pipeline_jobs.py` could also be split into ingestion, Silver, Gold, export and validation modules. Its current function boundaries provide a practical starting point.

4.9 Lessons from the implementation

The project showed why a reliable reporting pipeline needs both technical and semantic discipline. Operational and analytical systems serve different workloads. A watermark must only advance after the destination write succeeds. Raw, Silver and Gold represent different responsibilities. Spark performs the transformations, while Airflow controls their ordering.

The reporting boundary proved equally important. Stable column names, explicit grains and clearly defined measures are part of the interface with Tableau. Containerisation helps reproduce the runtime, but it does not replace validation or remove the need to understand the data. Together, these lessons connect the individual tools to the purpose of the platform.

General Conclusion

The internship produced a local data platform connecting Oracle records to Tableau reporting. Python captures source changes, MinIO retains Parquet batches, Spark builds validated and analytical layers, and Airflow coordinates execution. The result makes the path from a simulated insurance event to a reporting measure visible and repeatable.

The main contribution lies in how these stages fit together. Key and relationship checks, explicit analytical grains and aggregation before joining are as important as scheduling the jobs. Moving from notebooks to reusable functions turned an exploratory workflow into a documented execution path.

Synthetic data made the work possible without exposing customer information. It also defines the limits of the results: the dashboards do not describe Sanlam's actual portfolio, and the demonstrated volumes do not qualify an industrial deployment. The next improvements concern recovery, consistent publication, aligned runtime environments and reliable refresh of the Tableau sources.

The project brought together relational modelling, object storage, Spark processing, containerisation and business intelligence within one assignment. It provides a concrete foundation for studying how to operate and validate a data platform under more demanding conditions.

Glossary

Airflow DAG	A directed acyclic graph describing task dependencies and scheduling.
Bucket	A top-level logical container in an object store.
CDC	Change data capture: a mechanism for detecting source modifications, for example through transaction logs.
Gold	Business-oriented datasets prepared for analysis and reporting.
Grain	The meaning of one row in a dataset, such as one policy or one due month.
Hadoop-AWS	Libraries connecting Hadoop-compatible filesystem operations to S3 object storage.
MinIO	The local S3-compatible object-storage server used in the prototype.
Parquet	A column-oriented file format used for the lake's structured data.
Raw	The layer that retains extracted source batches.
S3A	The Hadoop filesystem connector used by Spark to access S3-compatible storage.
Silver	Consolidated, standardised and validated records.
Watermark	A stored boundary used to identify source rows that are eligible for the next extraction.

Bibliography

[1] Sanlam Maroc. *Company presentation and history*. https://sanlam.ma/fr/compagnie/. Accessed 11 September 2026.

[2] Sanlam Maroc. *Financial report at 31 December 2025*, company profile. https://sanlam.ma/media/uploads/BlockPublicationFinanciere/RAPPORT_FINANCIER_Sanlam_Maroc_Au_31_decembre_2025.pdf. Accessed 9 September 2026.

[3] Sanlam Maroc. *Integrated Report 2025*, pp. 11–15. https://sanlam.ma/media/uploads/BlockPublicationFinanciere/Rapport_Annuel_2025_BD_1.pdf. Accessed 11 September 2026.

[4] Allianz. *Newly launched SanlamAllianz to provide insurance and financial services in 27 African markets*, 5 September 2023. https://www.allianz.com/en/press/news/financials/stakes_investments/230905_Allianz-SanlamAllianz-to-provide-insurance-and-financial-services.result.html/9.html. Accessed 11 September 2026.

[5] SanlamAllianz. *Our Footprint*. https://www.sanlamallianz.com/en. Country list consulted 11 September 2026.

[6] Sanlam. *Sanlam reports strong performance in 2025 annual results*, 12 March 2026. https://www.sanlamonline.co.za/knowledge-hub/sanlam-reports-strong-performance-in-2025-annual-results. Accessed 11 September 2026.

[7] Natural Earth. *Admin 0 Countries, 1:110m; public-domain map data*. https://www.naturalearthdata.com/about/terms-of-use/. Geometry from the Natural Earth vector repository. Accessed 11 September 2026.

[8] Databricks. *What is the medallion lakehouse architecture?* https://docs.databricks.com/aws/en/lakehouse/medallion. Accessed 9 September 2026.

[9] Apache Software Foundation. *Apache Parquet: Overview*. https://parquet.apache.org/docs/overview/. Accessed 9 September 2026.

[10] Amazon Web Services. *Creating tasks for ongoing replication using AWS DMS*. https://docs.aws.amazon.com/dms/latest/userguide/CHAP_Task.CDC.html. Accessed 9 September 2026.

[11] Amazon Web Services. *Logical architecture of modern data lake-centric analytics platforms*. https://docs.aws.amazon.com/whitepapers/latest/aws-serverless-data-analytics-pipeline/logical-architecture-of-modern-data-lake-centric-analytics-platforms.html. Accessed 12 September 2026.

[12] Amazon Web Services. *Data lake foundation: Storage Best Practices for Data and Analytics Applications*. https://docs.aws.amazon.com/whitepapers/latest/building-data-lakes/data-lake-foundation.html. Accessed 12 September 2026.

[13] Apache Software Foundation. *Spark SQL and DataFrames*, Spark 4.0.1 documentation. https://spark.apache.org/docs/4.0.1/sql-programming-guide.html. Accessed 9 September 2026.

[14] Apache Software Foundation. *Dags*, Airflow 3.3.1 documentation. https://airflow.apache.org/docs/apache-airflow/3.3.1/core-concepts/dags.html. Accessed 9 September 2026.

[15] Docker. *What is Docker Compose?* https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-docker-compose/. Accessed 9 September 2026.

[16] Apache Software Foundation. *Hadoop-AWS module*, Hadoop 3.4.1 documentation. https://hadoop.apache.org/docs/r3.4.1/hadoop-aws/tools/hadoop-aws/index.html. Accessed 9 September 2026.